Heap Sort

Data Structures Using C – Sorting Algorithms

Department of Computer Science and Engineering

Graphic Era (Deemed to be University), Dehradun

Recall: Binary Heap

A binary heap is a complete binary tree that satisfies the heap-order property.

Max Heap

Every parent is greater than or equal to its children.

$$Parent \geq Children$$

Min Heap

Every parent is less than or equal to its children.

$$Parent \leq Children$$

For ascending Heap Sort, we use a max heap.

What is Heap Sort?

Heap Sort is a comparison-based sorting algorithm that uses the binary heap data structure.

- It first constructs a heap.
- It repeatedly removes the root.
- The root contains the largest element in a max heap.
- Each removed maximum is placed at the end of the active array.

Main Idea

Build Max Heap $\rightarrow$ Extract Maximum $\rightarrow$ Repeat

Heap Sort: Two Phases

According to the textbook, Heap Sort consists of two phases.

Phase 1: Build Heap

Construct a max heap using all elements of the input array.

Phase 2: Repeated Root Deletion

Repeatedly remove the maximum element and place it at the last available position of the array.

Why Use a Max Heap?

In a max heap:

$$\boxed{\text{Root} = \text{Maximum Element}}$$

Suppose:

$$H = [90,\ 70,\ 80,\ 20,\ 40,\ 60]$$

The maximum value is:

$$90$$

After moving 90 to the last position, the remaining elements are rearranged to restore the max-heap property.

Heap Representation Using an Array

A complete binary tree can be stored efficiently in an array.

For zero-based indexing:

$$Parent(i) = \left\lfloor \frac{i-1}{2} \right\rfloor$$

$$Left(i) = 2i + 1$$

$$Right(i) = 2i + 2$$

Important

The formulas are valid only when the corresponding parent or child exists.

Example: Max Heap

Consider:

$$H = [90,\ 70,\ 80,\ 20,\ 40,\ 60]$$



- 90 is the root.
- 70 and 80 are children of 90.
- 20 and 40 are children of 70.
- 60 is a child of 80.

Max-Heap Property

For each valid child index $i > 0$:

$$H[Parent(i)] \geq H[i]$$

Example:

$$H = [90,\ 70,\ 80,\ 20,\ 40,\ 60]$$

$$90 \geq 70, \qquad 90 \geq 80$$

$$70 \geq 20, \qquad 70 \geq 40$$

$$80 \geq 60$$

Therefore, the array represents a valid max heap.

How Can We Build a Max Heap?

Two common approaches are:

1. Repeated insertion using upward restoration.
2. Bottom-up heap construction using downward restoration.

Repeated Insertion

Insert each element and restore the heap property by moving it upward.
Time: $O(n \log n)$.

Bottom-Up Construction

Restore heap order from the last internal node toward the root.
Time: $O(n)$.

Heap Insertion: Basic Idea

When a new element is inserted:

1. Place it at the next available array position.
2. Compare it with its parent.
3. If it is larger, move it upward.
4. Repeat until the heap property is restored.

This operation is called:

RestoreHeapUp

Worst-case time:

$$O(\log n)$$

RestoreHeapUp: Example

Initial heap:

$$[80,\ 50,\ 70,\ 20,\ 40]$$

Insert:

$$90$$

Append 90:

$$[80,\ 50,\ 70,\ 20,\ 40,\ 90]$$

Compare 90 with its parent 70.
Since:

$$90 > 70$$

move 90 upward.

RestoreHeapUp: Continued

After exchanging 90 and 70:

$$[80,\ 50,\ 90,\ 20,\ 40,\ 70]$$

Compare 90 with its parent 80.
Since:

$$90 > 80$$

move 90 upward again.

Final heap:

$$\boxed{[90,\ 50,\ 80,\ 20,\ 40,\ 70]}$$

RestoreHeapUp: Algorithm

RESTORE_HEAP_UP(H, index)

1. Store $H[index]$ in VAL.
2. While $index > 0$:
    1. Find parent index.
    2. If parent value is greater than or equal to VAL, stop.
    3. Move parent value downward.
    4. Set index to parent index.
3. Place VAL at the final index.

RestoreHeapUp: C Function

```c
void restoreHeapUp(int heap[], int index)
{
    int value = heap[index];

    while (index > 0)
    {
        int parent = (index - 1) / 2;

        if (heap[parent] >= value)
            break;

        heap[index] = heap[parent];
        index = parent;
    }

    heap[index] = value;
}
```

Deleting the Root of a Max Heap

The root contains the maximum element.
To remove it:

1. Exchange the root with the last element in the active heap.
2. Reduce the active heap size.
3. Restore the max-heap property from the root downward.

This operation is called:

RestoreHeapDown

RestoreHeapDown: Basic Principle

Suppose the root violates the heap property.

1. Compare the left and right children.
2. Identify the larger child.
3. Compare the current value with that child.
4. If the child is larger, move it upward.
5. Continue downward until heap order is restored.

Worst-case time:

$$\boxed{O(\log n)}$$

RestoreHeapDown: Example

Consider:

$$H = [90,\ 70,\ 80,\ 20,\ 40,\ 60]$$

Remove the maximum:

$$90$$

Move the last element 60 to the root:

$$[60,\ 70,\ 80,\ 20,\ 40]$$

Compare the root with its children.
The larger child is:

$$80$$

RestoreHeapDown: Continued

Current active heap:

$$[60,\ 70,\ 80,\ 20,\ 40]$$

Since:

$$80 > 60$$

exchange 80 and 60.

Restored heap:

$$\boxed{[80,\ 70,\ 60,\ 20,\ 40]}$$

The maximum element 90 has been removed from the active heap.

RestoreHeapDown: Algorithm

RESTORE_HEAP_DOWN(H, index, size)

1. Store the current element in *VAL*.
2. Identify the left child.
3. While a left child exists:
    1. Identify the larger child.
    2. If *VAL* is greater than or equal to the larger child, stop.
    3. Move the larger child upward.
    4. Continue at the child's position.
4. Place *VAL* at its final position.

RestoreHeapDown: C Function

```
1  void restoreHeapDown(int heap[],
2                       int index, int size)
3  {
4      int value = heap[index];
5      int child = 2 * index + 1;
6
7      while (child < size)
8      {
9          if (child + 1 < size &&
10             heap[child + 1] > heap[child])
11             child++;
12
13         if (value >= heap[child])
14             break;
15
16         heap[index] = heap[child];
17         index = child;
18         child = 2 * index + 1;
19     }
20
21     heap[index] = value;
22 }
```

Heap Sort: Worked Example

Consider:

$$A = [4,\ 10,\ 3,\ 5,\ 1]$$



We will sort this array in ascending order.

1. Build a max heap.
2. Repeatedly place the maximum at the end of the active array.

Phase 1: Build the Max Heap

Initial array:

$$[4,\ 10,\ 3,\ 5,\ 1]$$

Using repeated insertion:

$$[4]$$

$$[10,\ 4]$$

$$[10,\ 4,\ 3]$$

$$[10,\ 5,\ 3,\ 4]$$

$$\boxed{[10,\ 5,\ 3,\ 4,\ 1]}$$

The maximum value 10 is at the root

Phase 2: First Extraction

Current max heap:

$$[10,\ 5,\ 3,\ 4,\ 1]$$

Exchange root 10 with last active element 1:

$$[1,\ 5,\ 3,\ 4 \mid 10]$$

Restore heap order in the first four positions:

$$\boxed{[5,\ 4,\ 3,\ 1 \mid 10]}$$

10 is now in its final position.

Second Extraction

Current array:

$$[5,\ 4,\ 3,\ 1 \mid 10]$$

Exchange root 5 with last active element 1:

$$[1,\ 4,\ 3 \mid 5,\ 10]$$

Restore heap order:

$$\boxed{[4,\ 1,\ 3 \mid 5,\ 10]}$$

The sorted suffix now contains:

$$[5,\ 10]$$

Third Extraction

Current array:

$$[4,\ 1,\ 3 \mid 5,\ 10]$$

Exchange root 4 with last active element 3:

$$[3,\ 1 \mid 4,\ 5,\ 10]$$

The remaining heap already satisfies the max-heap property.

Updated array:

$$\boxed{[3,\ 1 \mid 4,\ 5,\ 10]}$$

Fourth Extraction

Current array:

$$[3,\ 1 \mid 4,\ 5,\ 10]$$

Exchange 3 and 1:

$$[1 \mid 3,\ 4,\ 5,\ 10]$$

Only one element remains in the heap.
The array is now completely sorted.

$$\boxed{[1,\ 3,\ 4,\ 5,\ 10]}$$

Heap Sort: Complete Trace

Stage	Array
Initial	4, 10, 3, 5, 1
Build heap	10, 5, 3, 4, 1
Extraction 1	5, 4, 3, 1, 10
Extraction 2	4, 1, 3, 5, 10
Extraction 3	3, 1, 4, 5, 10
Extraction 4	1, 3, 4, 5, 10

Final result:

$$\boxed{1,\ 3,\ 4,\ 5,\ 10}$$

Heap Sort: Key Observation

After each extraction:

- The maximum active element moves to its final position.
- The active heap becomes smaller.
- The sorted suffix becomes larger.

$$\underbrace{\text{Active Max Heap}}_{\text{Unsorted}} \quad | \quad \underbrace{\text{Sorted Suffix}}_{\text{Final Positions}}$$

This continues until the active heap contains only one element.

Heap Sort: Algorithm

HEAP_SORT(A, N)

1. Build a max heap using the elements of A.
2. Set $SIZE = N$.
3. While $SIZE > 1$:
   1. Swap $A[0]$ with $A[SIZE - 1]$.
   2. Decrease $SIZE$ by 1.
   3. Restore the max-heap property from the root.
4. Stop.

Heap Sort: C Function

```c
void heapSort(int a[], int n)
{
    // Phase 1: Build max heap.
    for (int i = 1; i < n; i++)
        restoreHeapUp(a, i);

    // Phase 2: Extract maximum.
    for (int size = n; size > 1; size--)
    {
        int temp = a[0];
        a[0] = a[size - 1];
        a[size - 1] = temp;

        restoreHeapDown(a, 0, size - 1);
    }
}
```

Note

This implementation follows the textbook's repeated-insertion heap construction.

Complete C Program: Part 1

```c
#include <stdio.h>

#define MAX 100

void restoreHeapUp(int heap[], int index)
{
    int value = heap[index];

    while (index > 0)
    {
        int parent = (index - 1) / 2;

        if (heap[parent] >= value)
            break;

        heap[index] = heap[parent];
        index = parent;
    }

    heap[index] = value;
}
```

Complete C Program: Part 2

```
1  void restoreHeapDown(int heap[],
2                       int index, int size)
3  {
4      int value = heap[index];
5      int child = 2 * index + 1;
6
7      while (child < size)
8      {
9          if (child + 1 < size &&
10             heap[child + 1] > heap[child])
11             child++;
12
13         if (value >= heap[child])
14             break;
15
16         heap[index] = heap[child];
17         index = child;
18         child = 2 * index + 1;
19     }
20
21     heap[index] = value;
22 }
```

Complete C Program: Part 3

```c
void heapSort(int a[], int n)
{
    for (int i = 1; i < n; i++)
        restoreHeapUp(a, i);

    for (int size = n; size > 1; size--)
    {
        int temp = a[0];
        a[0] = a[size - 1];
        a[size - 1] = temp;

        restoreHeapDown(a, 0, size - 1);
    }
}
```

Explanation

The first loop builds a max heap.
The second loop repeatedly extracts the maximum element.

Complete C Program: Part 4

```c
int main(void)
{
    int a[MAX];
    int n;

    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1 || n < 1 || n > MAX)
    {
        printf("Invalid size.\n");
        return 1;
    }

    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++)
    {
        if (scanf("%d", &a[i]) != 1)
            return 1;
    }

    heapSort(a, n);

    printf("Sorted elements: ");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");
```

Sample Input and Output

Input

```
Enter number of elements: 5
Enter 5 elements:
4 10 3 5 1
```

Output

```
Sorted elements: 1 3 4 5 10
```

Heap Sort: Complexity of Heap Construction

Using repeated insertion:

$$T_{\text{insert}}(m) = O(\log m)$$

For n insertions:

$$T_{\text{build}}(n) = \sum_{m=1}^{n} O(\log m)$$

Therefore:

$$\boxed{T_{\text{build}}(n) = O(n \log n)}$$

Important

Bottom-up heap construction can build a heap in $O(n)$ time.

Complexity of Root Deletion

Deleting the maximum requires:

1. Swapping the root and last element.
2. Reducing the active heap size.
3. Restoring heap order downward.

The heap height is:

$$O(\log n)$$

Therefore, one extraction requires:

$$O(\log n)$$

For $n - 1$ extractions:

$$\boxed{O(n \log n)}$$

Overall Time Complexity

Phase 1:

$$O(n \log n)$$

Phase 2:

$$O(n \log n)$$

Therefore:

$$T(n) = O(n \log n) + O(n \log n)$$

$$\boxed{T(n) = O(n \log n)}$$

Heap Sort has:

$$\boxed{\Theta(n \log n)}$$

running time in the standard comparison model.

Best, Average and Worst Cases

Case	Time Complexity
Best	$\Theta(n \log n)$
Average	$\Theta(n \log n)$
Worst	$\Theta(n \log n)$

Observation

Heap Sort offers a guaranteed $O(n \log n)$ worst-case running time.

Heap Sort: Space Complexity

Heap Sort rearranges elements within the original array.

The presented implementation uses:

- A few integer variables.
- No temporary array.
- No recursive function calls.

Therefore:

$$\boxed{\text{Auxiliary Space} = O(1)}$$

Important

Heap Sort is an in-place sorting algorithm.

Is Heap Sort Stable?

Consider records:

$$[(2, A),\ (2, B),\ (1, C)]$$

The number is the sorting key.

The heap operations can exchange elements over long distances.
This may change the relative order of equal keys.

Correct Property

Standard Heap Sort is **not stable**.
This corrects the stability claim in the supplied textbook passage.

Properties of Heap Sort

Property	Heap Sort
Comparison-based	Yes
In-place	Yes
Stable	No
Adaptive	No
Best time	$\Theta(n \log n)$
Average time	$\Theta(n \log n)$
Worst time	$\Theta(n \log n)$
Auxiliary space	$O(1)$

Heap Sort vs Quick Sort vs Merge Sort

Property	Heap	Quick	Merge
Best time	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Average time	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Worst time	$O(n \log n)$	$O(n^2)$	$O(n \log n)$
Auxiliary space	$O(1)$	Recursion stack	$O(n)$
Stable	No	Usually no	Yes
In-place	Yes	Partition-based	No

Key Difference

Heap Sort combines constant auxiliary space with guaranteed $O(n \log n)$ worst-case time.

Advantages and Limitations

Advantages

- Guaranteed $O(n \log n)$ time.
- Constant auxiliary space.
- Suitable for large arrays.
- No recursive stack required.
- Uses efficient heap operations.

Limitations

- Not stable.
- Not adaptive.
- More complex than simple sorts.
- Often less cache-friendly than Quick Sort.
- Requires understanding of heaps.

Classroom Activity 1

Construct a max heap from:

$$[12,\ 7,\ 25,\ 15,\ 10]$$

Use repeated insertion.
Tasks:

1. Show the heap after each insertion.
2. Identify each parent-child comparison.
3. Identify the final root element.
4. Verify the max-heap property.

Classroom Activity 1: Solution

Insert elements one by one:

$$[12]$$

$$[12,\ 7]$$

$$[25,\ 7,\ 12]$$

$$[25,\ 15,\ 12,\ 7]$$

$$\boxed{[25,\ 15,\ 12,\ 7,\ 10]}$$

Root:

$$25$$

Classroom Activity 2

Apply Heap Sort to:

$$[20,\ 35,\ 15,\ 40,\ 10]$$

Show:

1. The constructed max heap.
2. The first root extraction.
3. The heap after restoration.
4. Every subsequent extraction.
5. The final sorted array.

Classroom Activity 2: Solution

Initial:

$$[20,\ 35,\ 15,\ 40,\ 10]$$

Max heap:

$$[40,\ 35,\ 15,\ 20,\ 10]$$

After first extraction:

$$[35,\ 20,\ 15,\ 10 \mid 40]$$

After second extraction:

$$[20,\ 10,\ 15 \mid 35,\ 40]$$

After third extraction:

$$[15,\ 10 \mid 20,\ 35,\ 40]$$

Final:

$$\boxed{[10,\ 15,\ 20,\ 35,\ 40]}$$

MCQ 1

Which element is stored at the root of a max heap?

1. Smallest element
2. Largest element
3. Middle element
4. Last inserted element

MCQ 1

Which element is stored at the root of a max heap?

1. Smallest element
2. Largest element
3. Middle element
4. Last inserted element

Answer

Option 2: Largest element.

MCQ 2

What is the worst-case time complexity of Heap Sort?

1. $O(n)$
2. $O(n^2)$
3. $O(n \log n)$
4. $O(\log n)$

MCQ 2

What is the worst-case time complexity of Heap Sort?

1. $O(n)$
2. $O(n^2)$
3. $O(n \log n)$
4. $O(\log n)$

Answer

Option 3: $O(n \log n)$.

MCQ 3

Which operation is performed after extracting the root of a max heap?

1. Binary search
2. RestoreHeapDown
3. Merge operation
4. Linear search

MCQ 3

Which operation is performed after extracting the root of a max heap?

1. Binary search
2. RestoreHeapDown
3. Merge operation
4. Linear search

Answer

Option 2: RestoreHeapDown.

MCQ 4

What is the auxiliary space complexity of iterative in-place Heap Sort?

1. $O(1)$
2. $O(n)$
3. $O(n \log n)$
4. $O(n^2)$

MCQ 4

What is the auxiliary space complexity of iterative in-place Heap Sort?

1. $O(1)$
2. $O(n)$
3. $O(n \log n)$
4. $O(n^2)$

Answer

Option 1: $O(1)$.

Common Programming Mistakes

1. Mixing zero-based and one-based indexing.
2. Using incorrect child index formulas.
3. Failing to select the larger child.
4. Restoring the heap over the sorted suffix.
5. Forgetting to reduce the active heap size.
6. Incorrect loop termination conditions.
7. Assuming standard Heap Sort is stable.

Practice Questions

1. Define a binary heap.
2. Differentiate max heap and min heap.
3. Explain the two phases of Heap Sort.
4. Explain RestoreHeapUp with an example.
5. Explain RestoreHeapDown with an example.
6. Trace Heap Sort on:

$$[4,\ 10,\ 3,\ 5,\ 1]$$

7. Write a C program for Heap Sort.
8. Derive its time complexity.
9. Explain why Heap Sort is not stable.

Lecture Summary

Key Concepts

- Heap Sort uses a binary heap.
- Ascending sorting uses a max heap.
- Phase 1 constructs the heap.
- Phase 2 repeatedly extracts the maximum.
- RestoreHeapUp supports insertion.
- RestoreHeapDown supports root deletion.
- Worst-case time: $O(n \log n)$.
- Auxiliary space: $O(1)$.
- Standard Heap Sort is not stable.

References

1. Reema Thareja, *Data Structures Using C*, Section 14.13: Heap Sort.
2. Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*.
3. Thomas H. Cormen et al., *Introduction to Algorithms*.

Lecture Feedback

Your Feedback Matters

Please share your feedback on:

- Binary heap concepts
- Heap construction
- Root deletion
- Heap restoration
- C implementation
- Complexity analysis



Scan to Submit Feedback

Thank You

Questions and Discussion

Build Heap $\rightarrow$ Extract Maximum $\rightarrow$ Restore Heap